ENBC321/BIOE658I: Homework #5

Due: Monday, 03/30/26 by 5:00 PM, beginning of the class

Submit your homework to ELMS, including Python code and the dataset

Overall points: 70

Note: Late homework may be submitted up to 24 hours after the due date, but a 50% penalty will be applied.

Note: Please aim to write clean, well-organized code that is easy for others to understand. Be sure to include comments or explanations where needed. Points may be deducted if this is not done.

Note: Please name your codes using the following format: "`problem1_fullname.ipynb`". Start with the problem number and followed by your full name, e.g., "**problem1_jimmyazarnoosh**".

Note: Run all cells and ensure there are no syntax errors. All results must be displayed in the notebook. (Syntax errors = zero points).

Note: Using generative AI for learning purposes is allowed. However, directly copying or closely replicating AI-generated content is strictly prohibited and will result in a grade of ZERO. Repeated violations may lead to more severe consequences.

Note: Submit **a single zip file** containing the following:

- The dataset you used
- Python code files in ***.ipynb**
- Your answers to the questions in PDF format

Problem 1. Gene and Protein Expression Classification using SVM (30 points)

You are given a dataset (**gene_expression_data.csv**) containing experimental values of gene expression and the corresponding protein expression levels, categorized into two classes: High and Low protein expression. Your task is to apply Support Vector Machine (SVM) Classification to **predict the protein expression class (High/Low)** based on gene expression.

The dataset contains the following columns:

Gene_Expression (X): Expression levels of a gene.

Protein_Expression (y): Categorical variable with two classes: High and Low indicating the measured protein expression levels.

Instructions:

- Load the dataset using **pandas**. The dataset is in a CSV file, so use `pd.read_csv()`.
- Preprocess the data:
  - Convert the **Protein_Expression** column to binary values: High = 1, Low = 0.
  - Create a threshold for determining whether the protein expression is high or low. let's assume that any Protein_Expression value **above the median** will be considered "High" (1), and values **below the median** will be considered "Low" (0).
- **Split the data** into training and testing sets (80% training, 20% testing).
- **Fit an SVM classifier** from the `sklearn.svm` module to the data.
- **Train the SVM classifier** on the training data.
- **Predict the protein expression class** on the test set.
- **Plot:**
  - The true protein expression values vs. the predicted values for the test set (use scatter plot).

Tasks:

- Compute the accuracy, precision, recall, and F1-score for the model.
- Use both **linear** and **nonlinear (RBF)** kernels and visualize each in a separate plot.
- For the **RBF kernel**, experiment with different C values: C = 0.01, 0.1, 1, 10, 100.
- Analyze how different Cs affect the model's performance (overfitting vs. underfitting).

Conceptual Questions:

1. How does the choice of kernel impact the classification performance of SVM?
2. Why is it important to normalize or scale gene expression data before training an SVM?
3. How does the C parameter influence the decision boundary of the SVM?

Problem 2. Classifying Tumor Types with Gaussian Naive Bayes (20 points)

You'll use the **Gaussian Naive Bayes (GNB) classifier** to classify tumors as malignant or benign based on features extracted from breast cancer cell samples. The dataset contains 569 samples, with 30 features describing characteristics like radius, texture, perimeter, area, smoothness, etc. These features are continuous, and you will apply Gaussian Naive Bayes to classify them into two classes: malignant (class 0) and benign (class 1).

Instructions:

- Load the Breast Cancer dataset from `sklearn.datasets.load_breast_cancer()`.
- **Split the data** into training and testing sets (80% training, 20% testing).
- **Fit a Gaussian Naive Bayes (GNB) classifier** from the `sklearn.naive_bayes` module to the data.
- **Train the Naive Bayes classifier** on the training data.
- Make predictions on the testing data.

Tasks:

- Compute the accuracy, precision, recall, and F1-score for the model.
- Use the `classification_report()` function from `sklearn.metrics` to get a detailed evaluation of the classifier.
- Plot a confusion matrix using `confusion_matrix()` and `heatmap()` from **seaborn** to better understand the classifier's performance across the classes.

Conceptual Questions:

1. Why does Gaussian Naive Bayes assume that the features follow a normal (Gaussian) distribution? How does this impact the classifier's performance when features don't follow a normal distribution?
2. How would the model's performance change if you used a different classification algorithm (e.g., Support Vector Machine, Logistic Regression)? Would Gaussian Naive Bayes still be appropriate for this problem?

Problem 3. Decision Trees vs. Random Forests (20 points)

Implement **Decision Tree** and **Random Forest classifiers** using **scikit-learn** to analyze their performance on a medical **Breast Cancer** dataset and answer conceptual questions about these models. The goal is to classify tumors as benign (0) or malignant (1).

Tasks:

- Use the Breast Cancer Wisconsin dataset from **scikit-learn**. `sklearn.datasets.load_breast_cancer()`
- Split the dataset into 80% training and 20% testing.
- Train a Decision Tree classifier (`DecisionTreeClassifier`) and a Random Forest classifier (`RandomForestClassifier` with 100 trees).
- Evaluate both models using **accuracy**, **precision**, **recall**, and **F1-score** on the test set.
- Train the Decision Tree with different values for `max_depth` (3, 5, 10) and `min_samples_split` (2, 5, 10). Analyze how these hyperparameters affect performance.
- Train the Random Forest with different values for `n_estimators` (10, 50, 200). Compare results.

Conceptual Questions:

1. Why might a Decision Tree overfit the training data?
2. How does Random Forest help reduce overfitting?
3. Compare the bias and variance of Decision Trees and Random Forests.
4. What happens when `n_estimators` (number of trees in a Random Forest) is increased?